



(c) 2025 Apollo — Real-Time Fire Monitoring System
Rights to make changes are reserved.
Subject to revision

Apollo Development Team
Daang Bakal Fire Station
Daang Bakal, Philippines

Repository: <https://github.com/migodbtc/apollo-mobile-application>
Contributors: @migodbtc, @hushiirei, Apollo Development Team
License: MIT

Contact: see repository for issue reporting and contact details



**Apollo: A Real-Time Fire Monitoring Mobile Application
Utilizing Machine Learning, Geographic Information,
and Alerts System For Daang Bakal Fire Station**

SYSTEM USER MANUAL (2025)



TABLE OF CONTENTS

System Requirements	1
API Interaction and User Authentication	2
API Payloads and Fields	2
Mobile UI Flow	3
Fire Report Submission and Response Tracking	4
Upload Endpoints	4
Background Verification and Data Flow	5
Notifications (SSE) and Mobile Behavior	6
Admin Dashboard Functions	7
Viewing and Managing Reports	8
Merging and Combined Reports	9
Troubleshooting Guide	10
Common problems and fixes	11
Contact and Support	20

SYSTEM REQUIREMENTS

To run the Apollo prototype locally, the following environment is recommended...

Operating System

- Windows 10 or 11 for development.
- Linux or macOS supported for server and web development.

Software Dependencies

- **Node.js**: Version 16.x or later (required for mobile tooling and web dashboard).
- **npm** (or **yarn**): Latest version compatible with Node.js.
- **Python**: Version 3.8 or higher (used by server and model code).
- **MySQL**: Version 5.7 or 8.0. A database named `apollo_system` is expected by the project SQL.
- **Expo CLI**: Recommended for mobile development.
- **Git**: Required for source control and version management.

**Configuration Notes:**

- The server is configured to connect to a local MySQL instance by default. Update `server/config.py` with your database credentials if necessary.
- The mobile app uses Expo and reads the API host from `expo.extra.SERVER_LINK` in `mobile/app.json` or from the runtime constant `mobile/constants/netvar.ts`.

API INTERACTION AND USER AUTHENTICATION

This section explains how the Apollo prototype communicates with the backend through API endpoints, including the structure of JSON payloads, the mobile app's registration and login flows, and security considerations for user authentication.

API Payloads and Fields

The Apollo prototype communicates through API endpoints defined in `server/main.py`.



Endpoints generally accept JSON POST payloads. Field names correspond exactly to database columns (e.g., `UA_user_id`, `UA_username`, `UA_password`, `UA_user_role`).

Examples

- `POST /user/add` — Creates a new user. Payload must include all required fields matching the DB columns.
- `POST /user/get/one` — Retrieves a single user. Example payload:

```
{ "UA_user_id": 1 }
```

Mobile UI Flow

- **Registration** **Page:**
`mobile/app/(auth)/register.tsx`
 - Builds JSON payloads according to server conventions.
 - Uses `AdminSQLContext` or project network utilities to call the API.



- **Login Page:** `mobile/app/(auth)/login.tsx`
 - Authenticates users.
 - Stores session context in `mobile/constants/contexts/SessionContext.tsx`

Security Note: Passwords are stored as plain text in this prototype. For production, always use hashed passwords and secure authentication tokens.

FIRE REPORT SUBMISSION AND RESPONSE TRACKING

Users submit fire reports through the mobile app. Each report is sent as a `multipart/form-data` POST request, containing:

- `report` — a JSON string with metadata (location, reporter ID, timestamp, etc.)
- **Exactly one file:** either image or video

Upload Endpoints

There are two main endpoints for uploading the report, with each route seen below:



`POST /reports/upload/image`

- Expects `multipart/form-data` with:
 - `report`: JSON string
 - `image`: binary file

`POST /reports/upload/video`

- Same format as above, but with `video` as the file field

Background Verification and Data Flow

The purpose of the background verification and data flow is to asynchronously verify incoming mobile reports (image/video) with the Hermes ML model, persist verification results, and notify clients in real time. Key files involving this process include `main.py` (routes + background thread), `inference.py` (HermesModel), DB tables `preverified_reports`, `postverified_reports`, `media_storage`, and `_layout.tsx` (SSE client + notifications).

The mobile app uploads a report with its media file, and the server quickly stores the blob, logs a pending entry in `preverified_reports`, and returns a fast response. A



background verifier periodically claims pending items, loads their media, runs the Hermes model, saves the full output in `postverified_reports`, updates the original row's status, and optionally cleans up blobs while sending an SSE notification. The client listens to the SSE stream, merges the updated verification data, and shows the user the final result.

Notifications (SSE) and Mobile Behavior

The purpose of the notifications is to deliver near-real-time verification and report updates from the server to mobile clients using Server-Sent Events (SSE). SSE is used for live UI updates and to trigger local notifications while the app is running. For background/terminated delivery, use push notifications (Expo/FCM/APNs).

- Endpoint: `GET /notifications/stream (text/event-stream)`. Mobile clients should connect after authentication and keep a single `EventSource` per session.



The mobile behavior focuses on capture → preview → upload flows, real-time SSE notifications, and local UI caching/merge logic: the app reads its server host from `expo.extra.SERVER_LINK` (`mobile/app.json` via `mobile/constants/netvar.ts`), captures media with the camera UI, posts a `multipart/form-data` report (a JSON "report" string plus exactly one file field named "image" or "video") to the server upload endpoints, then listens to the server SSE stream to receive verification updates which are merged into combined reports by `AdminSQLContext` and surfaced to users via Expo Notifications; background/terminated delivery of verification requires push notifications (not SSE), and developers must keep API shapes and SQL queries in sync across server and clients.

ADMIN DASHBOARD FUNCTIONS

The admin UI lives under `web/src/components/AdminComponents` (see `AdminDocumentationPage.tsx` for examples and patterns). The admin UI lists reports by reading the server endpoints



implemented in `main.py` and merging server results into the UI. Admins have access to both the mobile responsive version and the admin dashboard version, where the former is focused on portability and ease of access while the latter focuses on the full control and functionality of the administrative roles.

Viewing and Managing Reports

The admin dashboard presents a searchable, paginated list of incoming reports (preverified entries) with key columns—report ID, timestamp, location, reporter, media type, and verification status—and a detail panel that streams media from the server, shows full preverified and postverified metadata, and exposes admin controls to claim, annotate, reassign, override, delete/archive, or re-run verification; all management actions call the Flask API handlers in `main.py` using the project's JSON conventions (fields named exactly as DB columns) so changes must update server and all frontend callers.



- Key admin actions: claim/assign, override verification status, re-run verification and add admin notes.
- Filters & search: date range, status, media type, reporter, assigned admin, and bounding-box / municipality geofilter
- Audit & provenance: preserve preverified and postverified rows, log admin changes, and surface history in the detail view for manual reconciliation.

Merging and Combined Reports

The admin dashboard constructs a single Combined Report record for each submission by deterministically merging the fast preverified row created at upload with any later postverified ML outputs so admins can see the original submission and the verified result in one view; the merge matches rows by the report primary key or `media_id`, prefers postverified verification fields when present (`model_label`, `confidence`, `derived_status`) while preserving the original preverified metadata, updates combined entries in-place when SSE or polling delivers postverified updates,



and preserves all historical rows and admin overrides for audit and manual reconciliation rather than silently overwriting conflicting fields.

- Merge rules: join preverified and postverified by report_id/media_id; show postverified verification fields when available, keep preverified data under pre_metadata, and flag conflicts for manual review instead of auto-overwriting.
- UI & sync: render combined list from preverified rows immediately, then update entries in-place on SSE notifications or refetches; provide visual confidence indicators and an admin override that updates admin_status via API.

Audit & recovery: store and display full preverified/postverified histories, log admin actions (who/when/what), and offer re-run or manual reconciliation workflows to resolve metadata conflicts.

TROUBLESHOOTING GUIDE

This troubleshooting guide catalogs common runtime problems, how to quickly triage them, and concrete fixes



with repository pointers and commands; follow the checks in order (client → network → server → DB) and use server logs plus the mobile Metro/Expo logs to correlate failures. Below are short descriptions of each common issue followed by targeted diagnostic steps and fixes.

Common Problems and Fixes

The utilization of the system incurs common problems, and its associated fixes can be found here.

1. Axios error / cannot retrieve data from database (network or DB).
 - a. Symptoms: API calls from mobile/web return network errors, timeouts, or axios "Network Error".
 - b. Quick checks & fixes:
 - Verify `SERVER_LINK` used by clients: `netvar.ts` and `app.json` (`expo.extra.SERVER_LINK`) and `web/constants/netvar.ts`.
 - Confirm server is running: in server/ activate `venv` and start server:



- `cd server`
 - `.venv\Scripts\activate`
 - `python main.py`
 - Check server console for bind/port errors and exception stack traces (`server/main.py`).
 - Check MySQL connection: `config.py` credentials, and that `apollo_system` DB is imported (`sql/apollo_db_v1.0.3.sql`).
 - Ensure the machine LAN IP is used (not localhost). For Expo: use LAN IP or Tunnel.
 - If behind a proxy/firewall, open port (default 5821) or use Expo tunnel.
2. Admin dashboard not updating properly (UI stale; reenter sidebar tab refresh workaround)
- a. Symptoms: New/updated reports don't appear in the dashboard until manual refresh or reentering tab.
 - b. Quick checks & fixes:

- Confirm server returned updated data: call endpoint with curl/Postman (`curl -X POST http://<SERVER>:5821/reports/get/all -H "Content-Type: application/json" -d '{}'`)
- Ensure frontend triggers fetch on focus/visibility change (React Router / tab change). In mobile/web, add re-fetch in `onFocus/onShow` or use page lifecycle hooks.
- Verify `AdminSQLContext.fetchX()` is called after mutating operations and that `combineReports()` is used to merge pre/post sets.
- Check response shape: compare server response fields with what UI expects (names must match context types).
- Check caching layers: browser service worker or HTTP caching



may serve stale JSON — disable cache during dev or use cache-busting query param.

- Look for silent errors in browser/devtools console — network failures or JSON parse errors can abort updates.
- If SSE/Websocket is used for notifications, check connection logs and server endpoint `/notifications/stream` for dropped connections. Reconnect logic recommended.

3. Map too long to load (tiles or geoloading API)

- a. Symptoms: Map tiles take long or never load; loading state stays; map tiles render with error.
- b. Quick checks & fixes:
 - Open Network tab (browser / Expo devtools) and inspect tile requests: status codes, size, errors.



- Validate map API key or tile URL (typo or expired key causes 401/403).
- Reduce initial zoom and remove heavy overlays for faster startup.
- Implement progressive loading: show low-res tiles then higher detail.
- Cache tiles locally where possible or use a tile CDN.
- For long geocoding calls, move to server-side proxy or increase server timeout; use smaller batch queries.
- Locally test tile endpoints with curl to ensure reachable

4. Report stuck on submitting (stuck on report submission page)

- a. Symptoms: Mobile shows "sending" indefinitely; axios reports "Network Error" or timeout.
- b. Quick checks & fixes:



- Verify client `SERVER_LINK` (`mobile/constants/netvar.ts` and `app.json` `expo.extra.SERVER_LINK`) points to the correct server (use LAN IP not localhost when testing on device).
- Check server is running and listening on expected port (`.venv\Scripts\activate` && `python main.py`)
- Confirm with netstat: `netstat -ano | findstr 5821`
- Test upload endpoint with curl (multipart):
 - `curl -v -F "report={\"UA_user_id\":1}\" -F "image=@C:\path\to\img.jpg" http://<SERVER>:5821/reports/upload/image`
- Check Flask limits:
 - `MAX_CONTENT_LENGTH` in `config.py` (limits request

- size). Increase in large videos.
- Web server (nginx, reverse proxy) `client_max_body_size`.
- Check MySQL packet/blob limits: `max_allowed_packet` must be larger than largest blob. Adjust `my.cnf` and restart MySQL.
- Inspect server logs for exceptions or tracebacks when upload arrives. Missing commit/close may leave transactions hanging.
- On the client, increase axios timeout and enable `onUploadProgress` to show progress. Example: `timeout: 120000`.
- If connection drops on a mobile network, use Expo Tunnel or test on the same LAN as the server.



- For large video uploads, consider chunked uploads or background upload with resumable protocol.
- 5. Map not centered at current location on start (GPS or permissions)
 - a. Symptoms: Map opens centered to default coords instead of user location; no permission prompt or permission denied.
 - b. Quick checks & fixes:
 - Check permission request flow (Expo Location or react-native-permissions):
 - For Expo: `await Location.requestForegroundPermissionsAsync()`
 - `const pos = await Location.getCurrentPositionAsync({})`
 - Ensure permission promise is awaited before rendering map center.
 - Verify Android/iOS manifest:



- Android:


```
android.permission.ACCESS_FINE_LOCATION
```

 present and location usage description in app.json for Expo.
- iOS:


```
NSLocationWhenInUseUsageDescription
```

 in app.json.
- If an emulator/device lacks GPS, emulator geolocation must be set.
- Add fallback: if permission denied or `getCurrentPosition` fails, show UI prompt and use last known location or a sensible default.
- Ensure `setState` is called on the main thread and the map component receives prop update; use key prop to force map re-render if needed.



CONTACT AND SUPPORT

If you have any concerns or inquiries regarding the Apollo system that are not tackled within the contents of this user manual, the markdown documentation found within the repository itself, or the web application documentation section, then you can contact the original development team using the contact information below.

E-mail Address (Lead Developer)	mjdm Bunda@dbtcmandalu yong.one-bosco.org
Phone Number	+63 0956 99 0057
Response Time	1-2 Days
LinkedIn (Lead Developer)	Miguel Justin